Relatório – Torneio de Álgebra Linear (CIN0138)

Equipe: Grupo N — *(preencher os nomes dos integrantes)*
Arquivo entregue: `principal.py`, com a função `principal(observada, mascara)`

1. O problema

Recebemos uma matriz com parte das entradas ocultas e precisamos estimar essas entradas. O erro é medido só nas posições ocultas pelo NRMSE (raiz do erro quadrático médio dividida pelo desvio-padrão dos valores verdadeiros). Um NRMSE igual a 1 equivale a chutar a média; quanto menor, melhor.

2. Ideia central

Em tabelas de dados reais as linhas costumam ser parecidas entre si: cada linha é (quase) uma combinação linear de poucas linhas "básicas". Em termos de Álgebra Linear, a matriz tem **posto baixo**: o espaço das colunas tem dimensão pequena `r`, e a matriz pode ser escrita como um produto

$$X \approx U\,V^{\top}, \quad U \text{ com } m \text{ linhas e } r \text{ colunas}, \quad V \text{ com } n \text{ linhas e } r \text{ colunas}.$$

Uma matriz de posto `r` tem só $r(m + n - r)$ graus de liberdade, bem menos do que $m \cdot n$ entradas. Por isso as entradas conhecidas podem ser suficientes para determinar `U` e `V`, e o produto $U\,V^{\top}$ devolve as entradas ocultas.

Confirmamos isso nos gabaritos públicos calculando os valores singulares (SVD): 8 das 10 matrizes têm de 1 a 6 valores singulares grandes e o resto perto de zero.

3. O algoritmo, passo a passo

Passo 0 – preparação. A máscara chega como 0/1 e é convertida para booleana; as ocultas chegam como NaN e são trocadas por zero (nunca entram nas contas). Normalizamos a matriz: subtraímos a média μ das conhecidas e dividimos pelo desvio σ. No fim desfazemos a normalização.

Passo 1 – modelo simples (efeitos). Ajustamos $x_{ij} \approx \mu + a_i + b_j$ por mínimos quadrados alternados: com os b_j fixos, o melhor a_i é a média dos resíduos da linha i; depois fixamos os a_i e atualizamos os b_j. Uma pequena regularização λ no denominador evita exagerar o efeito de linhas com poucas entradas conhecidas. É o plano B do algoritmo.

Passo 2 – posto baixo por mínimos quadrados alternados (ALS). Queremos `U` e `V` que minimizem o erro nas entradas conhecidas (conjunto Ω):

$$\text{soma em } (i,j) \in \Omega \text{ de } (x_{ij} - u_i \cdot v_j)^2 \;+\; \lambda \text{ (penalidade em U e V)}$$

Com `V` fixo, cada linha u_i é a solução de um problema de mínimos quadrados com `r` incógnitas. As equações normais são

$$(V_i^{\top} V_i + \lambda\, c_i\, I)\, u_i = V_i^{\top} x_i$$

onde V_i contém só as linhas de `V` das colunas conhecidas na linha i, x_i os valores conhecidos e c_i quantos são. Resolvemos esse sistema $r \times r$ para cada linha (`numpy.linalg.solve`), depois trocamos os papéis e resolvemos para cada coluna, e repetimos. O ponto de partida vem da SVD da matriz com zeros nas ocultas. O termo $\lambda\, I$ (regularização *ridge*) garante que o sistema tenha solução única e estável mesmo com poucas entradas conhecidas.

Ajustamos o posto baixo em três versões da matriz, porque não sabemos de antemão qual descreve melhor os dados: (a) centralizada, (b) resíduo depois de tirar os efeitos do passo 1, (c) sem centralizar. Centralizar pode aumentar o posto em 1, o que faz diferença quando há poucas entradas.

Passo 3 – escolha do posto e da regularização por validação cruzada. O posto certo muda de matriz para matriz, e errar por 1 pode multiplicar o erro por 100. Como não temos o gabarito, simulamos o problema dentro das entradas conhecidas: dividimos as conhecidas em `K` grupos (10 nas matrizes menores, até 5 nas maiores); para cada grupo,

escondemos esse grupo, ajustamos o modelo com o resto e medimos o erro no grupo escondido. Fica o posto `r` (de 1 a 10) e o λ de menor erro. Os grupos são formados por uma fórmula fixa, sem aleatoriedade, então o resultado é sempre o mesmo. Os `K` ajustes são feitos de uma vez só com operações em lote do NumPy, o que mantém o tempo baixo.

Para cada posto percorremos um caminho de regularização da mais forte para a mais fraca (λ = 0,3; 0,1; 0,03; 0,01; 0,001; 0,0001), aproveitando a solução anterior como ponto de partida.

Passo 4 – trava contra previsões absurdas. Com λ muito pequeno o ajuste é quase exato, mas uma linha com poucas conhecidas pode gerar um sistema mal condicionado e uma previsão absurda. Por isso a previsão com λ menor não pode se afastar mais do que 3 erros típicos da previsão anterior (com λ maior, mais estável). O erro típico é o medido na validação.

Passo 5 – mistura e proteção para matrizes sem estrutura. A estimativa final é $w_1 \cdot$`(efeitos) + `$w_2 \cdot$`(posto baixo)`, com os pesos calculados por mínimos quadrados nas entradas escondidas da validação (pesos entre 0 e 1). Um modelo mais complexo só entra se o ganho sobre o mais simples for claro (teste *t* pareado maior que 2). Se nada ganhar da média, devolvemos a média: é o que acontece quando a matriz não tem estrutura aproveitável.

Passo 6 – segurança. As estimativas são limitadas a um intervalo um pouco maior que o dos valores conhecidos, qualquer valor não finito é substituído, os valores conhecidos são recolocados e, se ocorrer qualquer erro numérico, o código devolve o modelo de efeitos (ou a média). Assim a saída nunca tem NaN, infinito ou formato errado.

4. Resultados no banco público

NRMSE por matriz (avaliador local, mesma fórmula do torneio). As cinco primeiras colunas são as estratégias do laboratório do site da disciplina.

#	tamanho	ocultas	global	linha	coluna	efeitos	vizinhas	nossa
01	8x8	16%	1,234	1,171	1,368	1,264	1,310	**1,234**
02	9x6	13%	1,051	0,031	1,162	0,033	0,043	**0,034**
03	6x9	26%	1,099	1,269	0,074	0,077	0,371	**0,076**
04	14x5	13%	1,039	0,580	0,873	0,303	0,231	**0,041**
05	14x7	17%	1,001	0,800	0,900	0,453	0,268	**0,162**
06	16x16	27%	1,031	1,086	1,053	1,115	0,906	**0,042**
07	18x22	26%	1,001	1,042	1,027	1,054	0,752	**0,088**
08	17x17	34%	1,003	1,030	1,015	1,066	1,208	**1,003**
09	24x19	35%	1,011	0,723	0,859	0,248	0,266	**0,001**
10	28x31	41%	1,001	0,829	0,730	0,327	0,335	**0,001**
média			1,047	0,856	0,906	0,594	0,569	**0,268**

Interpretação. As matrizes 4, 6, 7, 9 e 10 têm posto baixo (2, 3, 4, 4 e 6): a validação encontra o posto e o erro cai muito; nas 9 e 10 quase não há ruído e a reconstrução é praticamente exata. As matrizes 2 e 3 têm linhas ou colunas quase constantes (posto 1) e todos os bons métodos empatam. Na 5 (14x7, posto 3 com ruído) há poucas entradas para o número de parâmetros e o erro é maior. As matrizes 1 e 8 não têm estrutura de posto baixo (na 8 os valores singulares caem devagar, como em uma matriz aleatória): nenhuma estratégia fica abaixo de 1, e o algoritmo corretamente recua para a média em vez de piorar.

Para não ajustar o código só a essas 10 matrizes, testamos também em 304 matrizes geradas por nós (5x5 até 40x40, 10% a 50% ocultas, vários tipos de estrutura): NRMSE médio de **0,306**, contra 0,642 da melhor estratégia simples. O pior tempo em matrizes 40x40 foi de cerca de 0,3 s (limite: 2 s).

5. Limitações

- Matriz sem estrutura: não há o que recuperar; o resultado é a média (NRMSE ≈ 1).

- Linha ou coluna totalmente oculta: nada na matriz informa sobre ela; usamos o efeito da outra dimensão.
- Matrizes muito pequenas: com poucas entradas a validação cruzada é ruidosa e o posto escolhido pode não ser o ideal.

6. Referências

- G. Strang, *Linear Algebra* (MIT 18.06), aulas 15–16 (projeções e mínimos quadrados) e 29 (decomposição em valores singulares).
- Laboratório "Matriz oculta" e materiais do site da disciplina CIN0138.
- Google Developers, *Collaborative Filtering / Matrix Factorization*.
- R. Mazumder, T. Hastie, R. Tibshirani, "Spectral Regularization Algorithms for Learning Large Incomplete Matrices", JMLR, 2010.
- Y. Zhou et al., "Large-scale Parallel Collaborative Filtering for the Netflix Prize" (ALS com regularização ponderada), 2008.
- Documentação do NumPy: `numpy.linalg.svd`, `numpy.linalg.solve`.